Mastering Microsoft Azure: Advanced Services

Section 1: Introduction

Course Introduction

Join the Cloud and Software Architects Community

Get the Course Slides

Who is This Course For?

Agenda

Section 2: Quick Introduction to the Cloud

Read This before Going Through This Section

Current Status in Computing

What is the Cloud?

Characteristics of the Cloud

IaaS, PaaS, SaaS

Types of Clouds

Main Cloud Providers

Introduction to Azure

Regions and Zones

Azure Services

Instantiating an Account

Section 3: Preparing the Environment

Read This Before Going Through This Section

Install .NET Core SDK

Install VS Code

Install VS Code Extensions

Section 4: IOT Hub

Introduction

1. IOT Hub:
    1. It is a central messaging hub for bi-directional communication with IOT devices
    2. It can handle millions of devices
    3. It is extremely reliable and secure
    4. It supports multiple messaging patterns
        1. Important when designing
    5. It has built-in monitoring
2. IOT:
    1. Internet of Things
        1. It is hardware devices (=things) connected to the internet
            1. Examples:
                1. Thermostats
                2. Light bulbs
                3. Routers
                4. Refrigerators
                5. Cameras
                6. Switches
            2. The above devices connect to the internet
                1. Why?
                    1. Devices connect in order to...
                        1. Send telemetry - to know their status
                        2. Get softwar updates
                        3. Send images - cameras might want to send images of whats going on in our smarthome
                        4. Send sensor data - Whats the current temperature in our home
                        5. ...

2. There are many scenarios when we want devices to connect to the internet either to send data or to receive data
3. How do the devices connect to the internet
    1. Solution: IOT Hub
        1. It is a hub where devices connect to
            1. Hub is:
                1. Secure
                2. Reliable
                3. Bi-Directional
                4. Monitored

3. Device Connection to IOT Hub:
    1. Extremely simple code
        1. It can use Azure IoT Hub SDK
            1. We don't have to use the SDK but it will make life easier when connecting to IoT hub
            2. Supported in:
                1. C
                2. Python
                3. Node JS
                4. .NET
                5. Java
            3. Chances are that if we are a developer designing and developing an IoT system, then the language is covered by Azure IoT Hub SDK
4. Communication Protocols Supported (standard IoT Device protocols and a device may support at-least one of them):
    1. HTTPS
    2. AMQP
    3. MQTT

Message Routing

1. Important concept of IoT Hub

    1. Connecting to IoT Hub is not enough...
        1. Devices connect to the IoT Hub
        2. IoT Hub will get telemetry messages, images, ...
        3. Now What?
            1. What are we going to do with the data received from the IoT Hub?
                1. Solution: Message Routing

2. Message Routing

    1. Message from the IoT device should be routed somewhere
    2. With IoT hub we have a lot of options where exactly to route messages
        1. We have a lot of endpoints with IoT hub
            1. Message router can send messages to
                1. Event Hub
                2. Service Bus Topics
                3. Service Bus Queues
                4. Storage Blob
                5. Custom endpoint connectors - we can configure and develop
                6. Built-in endpoint - comes out of the box with IoT Hub
                    1. It is essentially an eventhub topic
    3. The endpoints are discussed in previous course

3. Typical IoT Hub Architecture

```
IoT Device
    |
    v
IOT Hub
    |
    v
Event Hubs
    |
    v
Function (Validation - waits for it, checks if everything is in place (params are valid))
    |
    v
Event Grid (Pub/Sub - publishes to other listeners)
    |
    v
Function (Insert into DB)
    |
    v
Cosmos DB
    ^
    |
App Service (Queries and shows it)
```

    1. It is a real-world architecture we can use for our own IoT systems
    2. Device uses: D2C (Device-to-Cloud) communication
        1. IoT device only sends messages to IoT Hub (not the other way around)

4. Device-to-Cloud Communication

    1. The classic communication pattern
        1. Device sends data in any shape or form to the cloud or IoT Hub
    2. Three modes of communication:
        1. Telemetry & Data (the most common pattern for sending device data to the cloud)
        2. Files upload
        3. Device Twin

5. Cloud-to-Device Communication

1. The other way around...
2. Used mainly for:
    1. Send commands to the device
        1. If IoT Hub or user at the cloud wants the device to do something, then it can send a command to the device
            1. Example:
                1. Take a picture
                2. Measure temperature
    2. Software updates
        1. Once in a while, the software in a device needs to be updated
            1. CTD communication can be used
3. It uses direct call to communicate with specific device(s)

Security

1. IOT Hub Security
    1. Security is one of the major issues with IoT
        1. Why?
            1. Need to make sure that:
                1. Only allowed devices can connect to the IoT Hub
                    1. We want to avoid a malicious device from connecting to IoT hub and poisons our cloud system
                2. Communication is safe
                    1. The data that was received by the IoT hub was not tampered with
2. Device Authentication
    1. Devices can authenticate using one of the following:
        1. SAS token
            1. We used SAS token when connecting to storage account
        2. Individual X.509 certificate (we have a certificate per device)
        3. X.509 CA authentication (single authentication issued for all the devices using a certified CA)
3. Secure communication:
    1. Communication to IoT Hub is doen using TLS
        1. The standard encryption protocol that is used for secure web communication
    2. We can configure the minimum TLS version that is allowed when connecting to the IoT Hub
        1. Note: v1.0 & v1.1 are deprecated
            1. Should not be allowed to connect to the IoT hub

Tiers and Pricing

1. Two Tiers:
    1. Basic
        1. Capabilities:
            1. Device-to-cloud telemetry
            2. Per-device identity
            3. Message routing, message enrichments, and Event Grid Integration
            4. HTTP, AMQP, and MQTT protocols
            5. Device Provisioning Service
            6. Monitoring and Diagnostics
    2. Standard (Have Free)
        1. Capabilities:
            1. Device-to-cloud telemetry
            2. Per-device identity
            3. Message routing, message enrichments, and Event Grid Integration
            4. HTTP, AMQP, and MQTT protocols
            5. Device Provisioning Service
            6. Monitoring and Diagnostics
            7. Cloud-to-device messaging
            8. Device twins, Module twins, and Device management
            9. Device streams (preview)
            10. Azure IoT Edge
            11. IoT Plug and Play
2. The basic differences between the basic tier and standard tier is the cloud to device messaging
    1. If we want to send messages from the cloud to the IoT device, we want to go to the standard tier
3. Standard tier supports Device twins
    1. If we need this capability, we cannot go with the Basic tier

Constructing and Setting up IOT Hub

1. Resource group for IoT Hub
    1. Search: Resource Groups
        1.
            - Create
            1. Resource group: IOTHub-rg
            2. Region: West Europe
            3. Review + Create
            4. Create
    2. Search: IOT Hub
        1.
            - Create
            1. Resource group: IOTHub-rg
            2. Region: West Europe
            3. IOT hub name: course-iot
            4. Management:
                1. Pricing and scale tier: Standard
                    1. We need the following features:
                        1. Cloud to device messaging
                        2. Device twin
            5. Review + Create

6. Create
7. Go to reasource

2. Built in endpoints
    1. Endpoints - places where IoT hub sends messages once it gets them from devices
        1. Built-in endpoint - EventHub Topic
            1. Endpoint of EventHub: Use the connection string to connect
    2. Messaging:
        1. Message routing: Mechanism that IoT Hub uses to route messages to custom endpoints
            1. Custom endpoints
                1. Types of supported endpoints
                    1. Storage
                        1.
                            - Add
                        1. Storage
                            1. Endpoint name: error-msgs
                                1. The endpoint will be used to report errors in the messages received by the IoT hub
                            2. Pick a Container - to store messages
                                1.
                                    - Storage account - dedicated for use by the IoT Hub
                                1. Name: iotmsgsmutthoju (unique)
                                2. Account kind: Storage v2 (modern type)
                                3. Replication: Locally-Redundant-Storage (no DR)
                                4. Location: West Europe
                                5. Minimum TLS version: Version 1.2
                                6. OK
                            3. Go to resource
                                1. Storage accounts
                                    1. iotmsgsmutthoju
                                        1.
                                            - Container
                                        2. Name: error-msgs
                                        3. Public access level: Private (no anonymous access allowed)
                                        4. Create
                                    2. Select error-msgs
                                        1. Batch Frequence: Max elapsed time from the moment the IoT Hub receives a message until it routes to our container (60s to 720s)
                                            1. 60s
                                        2. Chunk size window: Max size of blob written to the storage account:
                                            1. 100MB
                                    3. Encoding: JSON
                                        1. Clear text
                                            1. Avro - binary
                                    4. File name: default
                                        1. iothubname-partition-date
                                    5. Create
                        2.
                            - Add
                        1. Storage
                            1. Endpoint name: valid-msgs
                            2. Pick a container: iotmsgsmutthoju
                                1.
                                    - Container:
                                1. valid-msgs
                                2. Create
                            2. Select the new container
                                1. Batch frequency: 60s
                                2. Encoding: JSON
                                3. Create
            2. Routing messages to the endpoint
                1. Routes - to define routes so that IoT Hub knows which messages to route to with endpoint
                    1.
                        - Add
                    1. Name: error-route
                    2. Endpoint: error-msgs
                    3. Data source: Device Telemetry Messages (messages flowing from device to IoT hub containing data, telemetry, etc.)
                        1. Other types:
                            1. Lifecycle events
                            2. Connection state
                            3. Twin change
                            4. ...
                    4. Enabled
                    5. Routing Query: Allows us to define the contents of the messages so that they will be routed to the endpoints

                        1. true - every message will automatically be routed

                        2. Only error messages:

                        ```
                        $body.status='error'
                        ```

                            1. In the body, a field named status should have the value error (if field doesn't exist, the message is ignored)
                    6. Test:
                        1. Message body:

                            1. Type the following JSON

```
{
    "status": "error",
    "message": "error getting data from db"
}
```

1. Test route

2. Message matched the query

2.
   - Add

1. Name: valid-route

2. Endpoint: valid-msgs

3. Routing query:

```
$body.status='success'
```

4. Save

Adding Devices and Access Policies

1. IoT devices:
   1.
      - New - Register devices that are going to use the IoT hub
   1. Ways to do it:
      1. Manually
      2. Using API
      3. Using provisioning services
      4. ...
   2. Device ID: every device should have a unique id
      1. iotdev1
   3. Save
2. Refresh
3. Authentication type
   1. SAS
      1. It is a specialized kind of token that is automatically generated by Azure which provides authentication against Azure cloud
      2. We have to use specialized string (connection string) for the device to connect to IoT hub
   2. Shared access policies
      1. List of policies that define access scope of IoT hub - permissions of anyone using the policies
      2. iothubowner - must be used by owner only
         1. device must not use it
      3. device - used by device when connecting to IoT hub (for sending data)
         1. Click
         2. Primary and Secondary connections strings
            1. Permissions: Device Connect
      4. iothubowner
         1. Permissions: full suite
         2. copy primary connections string

A Note About IoT Extensions in VS Code

1. IoT related extensions in VS Code
2. Install the following instead of Azure IoT Tools:
   1. Azure IoT Hub
   2. Azure IoT Edge

Sending Messages to IoT Hub

1. VS Code
   1. Extensions
      1. Azure IoT Tools
         1. Install
2. Explorer
   1. AZURE IOT HUB
      1. Set IoT HUB Connection String
         1. Paste the connection string
         2. Connection string is updated
      2. Right click on iotdev1
         1. Send D2C Message to IoT Hub

            1. Device: iotdev1

            2. Messages per device: 1

            3. Data Template

```
{
    "status": "error",
    "message": "Failed to retrieve data"
}
```

            4. Send

      3. Duplicate portal:

1. Storage accounts
    1. iotmsgsmutthoju
        1. Containers
            1. error-msgs
                1. empty
                    1. Why?
2. Message routing:
    1. Disable fallback routes - IoT has a fallback route (default route)
        1. Built-in endpoint
            1. Any message that does not fall within the defined routes will go to fallback route
3. Metrics
    1. 1 message that was routed to fallback route
        1. Why?
            1. Built in endpoints (eventhub topic)
                1. Copy the connection string
                2. VS Code
                    1. Right click on device
                        1. Start Monitoring Built in Event Endpoints
                        2. Paste connection string
                            1. VS Code is monitoring built-in endpoint
                        3. Send
                        4. Change "error" to "success"
                        5. Send
                            1. Every message is routed to fallback route instead of custom endpoint
                                1. Why?
                                    1. Properties need to be set
                                        1. Cannot be set in a simulator
                                            1. Solution: We need code
                                                1. VS Code can take care of that

Using Code to Send Messages

1. VS Code:
    1. Right click device
        1. Generate Code
            1. C#
            2. Send device-to-cloud message
            3. Location: C:/iothubcode/d2c
        2. csproj
            1. Version of .NET core needs to be compatible with what is installed on the system
                1. 3.1
        3. vscode/launch.json
            1. `...netcoreapp3.1`
        4. Packages are still missing
            1. Terminal:

               ```
               dotnet add package newtonsoft.json -version 13.0.1
               ```
        5. F5
            1. Sends messages to IoT hub
                1. Connection string is pre-populated
2. Go to another instance of VS Code
    1. Monitor of built in endpoint
        1. Shows messages currently received on the other instance
3. Stop sending messages
    1. Code:
        1. `SendDeviceIoTCloudMessageAsync()` - sends device
        2. JSON message string
            1. Encoding for JSON
        3. Properties
        4. `SendEventAsync(...)` - sends messages to the IoT Hub
        5. `Main` - entry point
            1. Initialization
            2. Calls methods to send messages
        6. Change:
            1. Remove references of weather
            2. Remove comments
            3. Define message:

```
var messageString = "{'status':'error','message':'Error retrieving data'}";
```

7. F5
8. Messages are still received in fallback route
    1. Why?

Fixing the Code to Enable Routing

1. We need to set some properties
    1. Code:

```
message.contentType="application/json";
message.contentEncoding="utf-8";
```

    2. Go to monitor:
        1. Click bottom-right button to clear the monitor (it must be empty after sending messages)
    3. F5
    4. Monitor:
        1. It is empty - nothing is routing to fallback route
2. Portal:
    1. Metrics
        1. Metric: Routing - messages delivered to storage
            1. Refresh
                1. Messages were delivered to the storage
    2. error-msgs - container
        1. Refresh
            1. course-iot
                1. O2/////.json
                    1. A bunch of messages
                        1. Each line is a single message
                        2. Metadata + body
3. VS Code:
    1. Stop code from running
        1. Change "error" to "success"
        2. F5
    2. Nothing is routed to built-in endpoint
4. Portal:
    1. valid-msgs - container
        1. It takes some time to write messages (depending on the frequency parameter)
        2. Refresh
        3. O2/////.json
        4. Message contains the body

Sending Cloud-to-Device Messages

1. How to send messages from IoT Hub to devices
    1. Sending commands
    2. Updating software
    3. ...
2. VS Code
    1. Right click on the device
        1. Start Receiving C2D Message
            1. VS Code will listen to messages from the cloud
3. Portal
    1. IoT devices
        1. iotdev1
            1. Message to device
                1. Message body:

```
hello fron IOT Hub.
```

                2. Send Message
4. VS Code
    1. Message appears
5. Using Code:
    1. Download resource
        1. Paste contents in `Send...Device.cs`

```
private static async void RecieveC2DAsync() {
    ...
    ...receivedMessage.GetBytes()...
    ...
}
```

            1. If the received message is not null, we print it to console
        2. In `Main` method, paste the following method call:

```
RecieveC2DAsync();
```

3. Click: Stop Recieving C2D Message (bottom)
    1. Or else, monitor will receive the message instead of the code
4. F5

2. Portal:
    1. Send Message
        1. Message gets printed
3. Try with a different message

6. How to send messages from code?
    1. Portal:
        1. Shared access policies
            1. Open `service` policy - it is code that sends messages to IoT hub
                1. Service Connect permission
                2. Copy Primary connection string
    2. VS Code

```
private const string s_serviceConnectionString = "<paste-connection-string>";
// connection string will be in a different code file
// For production, save the connection string in Key-Vault
```

    3. Download resource file
        1. Copy the content and paste above `Main`

```
private async static Task SendCludDeviceMessageAsync() {
    ...
}
```

            1. Terminal:

```
dotnet add package Microsoft.Azure.Devices
```

            2. Comment out the method we don't need
            3. Point to the error: Ctrl + point
                1. Microsoft Azure Devices
                2. Microsfot Azure Device Messages
            4. `Main`

```
SendCloudToDeviceMessageAsync();
```

                1. Ctrl+Point
                    1. Second option
            5. Right click on iotdev1 device
                1. Start Receiving C2D Messages
            6. Run the code: F5
            7. Output
                1. Message was received
            8. Try it again by changing message
    4. Code:
        1. We are now using service client (because we are sending messages)
        2. We specify a target device
        3. `SendAsync` is used to send a message

Device Twins

1. What is device twins?
    1. It is a JSON document that reflects the current state of the device
    2. It is stored in the IOT Hub
2. Why do we need it?
    1. It can be used to:
        1. Allows the IOT Hub to change device state
            1. If we modify the JSON document, the change immediately reflects in the device
        2. It allows us to report back to the IOT Hub on changes in device state
            1. When a state of the device is changed, the changes immediately reflects in the JSON document
                1. IOT Hub is notified that something was changed
        3. IOT Hub recieves notifications on changes
        4. IOT Hub can query the devices based on the state
            1. It can get a set of devices that meet a specific state
                1. Example: All the devices that are in the non-active status
3. Device Twin:

1. Preset sections:
    1. Device Identity -
        1. Status
        2. Update time
        3. Connection status
        4. (some data about device activity, and identity)
    2. Tags: data about the device set by the back end
        1. Back end: IoT Hub and all endpoints it routes to
            1. Tags are invisible to device
                1. Device doesn't have an idea about deployment location
    3. Desired Properties: Set by the backend & read by the device
        1. The way for backend to signal the device, what it wants the device to do or what it wants the device to be
            1. Example: send frequency = 5m
    4. Reported Properties: Set by the device, read by the backend
        1. Example: send frequency = 5m (actual send frequency) -> status = success
            1. If the send frequency is not 5m, the device sends a failure
                1. Backend should be aware of failure and act accordingly
2. Functionality of each section:

```
Device App                                                          Backend

                  Device Twin
                  - Tags                            Read, write
                                          change notifications
                  - Properties
Read, receive                 - Desired             Read, write
change                                                              change
notifications                                                 notifications

Read, write                             - Reported              Read
                                         change
                                         notifications
```

3. Usecases:
    1. Very useful for changing state of devices
        1. If changes occure too frequently, IOT Hub might throttle the changes and merge them
            1. Solution: Use regular D2C messages (telemetry data)
        2. IOT Hub is designed to cope with large amounts of telemetry data:
            1. Device twins is not a replacement for such data

Using Device Twins

1. VS Code
    1. Right click iotdev1
        1. Edit Device Twin
            1. Basic device twin automatically setup
                1. ID of device
                2. Properties
                    1. Desired properties
                    2. Reported properties
            2. Download the resource
                1. Unzip
                2. Sets tags and properties in a device twin of a specific device

```
static string connectionString = '<paste-connection-strin-with-service-permission>'; // manages devices but doesn't repor

var twin = await Task SetDeviceTags() {
    var twin = await
}
```

Device Provisioning Service

1. Device Provisioning Service (DPS)
    1. Connecting and configuring 100Ks (or even more...) of devices can be a long & tedious task
        1. Each device should be:
            1. Configured to connect to the IOT Hub
            2. Configured to have the desired properties (Device Twin - if using it)
    2. Solution: DPS
        1. Manufacturer sets the connection to the DPS (manual)
            1. It is part of the manufacturing process
                1. It is written to the hardware itself
        2. Sys admin sets the list of devices allowed to connect and the IOT Hub they connect to (manual)
        3. Devices connect to the DPS (when turned on), then to the IOT Hub, and automatically gets configured (auto)
            1. Connecting to IOT Hub, and getting desired properties of device twin in done automatically
2. Flow:
    1. Device Provisioning Service
        1. Admin sets the list of devices and configurations in teh DPS
            1. Prepares a list of devices allowed to be connected to DPS
            2. How should the devices be connected
    2. Device connected to DPS (using parameters set by manufacturer)
        1. Happens when device is turned on
    3. DPS validates the device
        1. Ensures that the device appears in the enrollment list (defined in step 1)
    4. DPS registers the device with appropriate IOT Hub

        1. Uses API - automatically
    5. IOT Hub returns the device ID which was generated automatically to the DPS
    6. The DPS returns the connection information to the device
    7. Device connects to the IOT Hub using the connection information
    8. Device gets desired configuration using device twin
3. DPS features:
    1. DPS supports up to 1,000,000 devices (soft limit)
        1. If we want more, we can ask Microsoft to increase the limit
    2. DPS is easily configurable from the Azure Portal
    3. DPS is extremely cost effective
        1. IoT HUb Device Provisioning Service:
            1. 1000 (1 million) x $0.10 = $100.00 / month

More About IOT Hub

1. SLA: 99.9% (excludes Free tier - no SLA)
2. Message size for calculating the number of messagtes is 4K
    1. Larger messages are split (two or more messages)
3. Devices can be bulk-imported using the `ImportDevicesAsync` of the `RegistryManager` class
4. IOT Hub can be secured and connected to VNet using Private Link
    1. It exposes private IP to the VNet
        1. Miminizes attack surface
5. DR is achieved using MS-initiated failover (RTO (Recovery Time Objective) of 2-26 hours) or manual failover
    1. MS triggers failover to another region
        1. Manual trigger has shorter RTO
        2. It gets triggered when we want it to be triggered
        3. We will know when MS will trigger

Resources

1. IOT Hub is an advanced service with a lot of capabilities
    1. To learn more:
        1. Securing IOT Hub with VNets and Private Links: https://docs.microsoft.com/en-us/azure/iot-hub/virtual-network-support
        2. High availability and DR: https://docs.microsoft.com/en-us/azure/iot-hub/iot-hub-ha-dr
        3. Monitoring IOT Hub, including logging: https://docs.microsoft.com/en-us/azure/iot-hub/monitor-iot-hub
        4. Message enrichment: https://docs.microsoft.com/en-us/azure/iot-hub/tutorial-message-enrichments

Section 5: API Management

Introduction

1. What is API Management
    1. It is a central API gateway that manages and organizes API access to backend apps
    2. It manages the:
        1. API
        2. Monitoring
        3. Rate limit
        4. Quota
        5. Security
        6. Caching
        7. ...
    3. It allows defining various policies for the APIs
    4. It is an extremely flexible service
    5. It is easy to manage and maintain
2. It is like a front-end that stands in front of backend apps and manages the access to the apps
3. API Management deployment:

```
API Management
    |
    |
    +----------------+----------------+
    |                |                |
On-Prem          Azure            Azure
Backend App #3   Bakend App #2    Backend App #1
Java, SOAP       NodeJS, REST API .NET, REST API
```

    1. Each app is developed using different technologies
    2. API management unifies access to the apps and centralizes all the access aspects to the APIs
4. Three main components:
    1. API Gateway
        1. It accepts the actual API calls and routes to backend APIs
            1. While doing that, it validates credentials
            2. It enforces quotas and rate limits
            3. It implements policies (if policies were defined)
            4. It caches responses where relevant
            5. It logs the calls for future analysis
        2. It sits in front of the apps and does the hard word
    2. Azure Portal
        1. We define the API that API management is going to manage and expose
        2. We can package APIs into products
        3. We can set up policies:
            1. Quotas

2. Rate limit
3. ...
4. We can get insights from usage data
5. We can manage users' access to the API management

3. Developer Portal
    1. It is a specialized portal which is a completely separate portal
    2. It is for developers that are going to use the APIs exposed by API management
        1. What can they do in the portal?
            1. Read API docs
            2. Test APIs
            3. Subscribe to APIs
            4. Get your own API usage data
                1. How many times did the developer use the API
                2. ...

5. API Management Terminology:

    1. Operation
        1. It is a specific operation in the API
            1. Usually - a specific URL in REST API
                1. Example: Managing users API
                    1. Adding user: POST /api/user
    2. Product
        1. A logic container of one or more APIs with usage quotas and terms of use
            1. Example: User management API
                1. We have a product for managing users
                    1. Contents of product:
                        1. API for accessing user's data
                        2. API for getting usage analysis
                        3. ...
                            1. The APIs will be packaged in a single product
    3. Rate limit
        1. A limit to the number of requests an API can accept.
            1. It is used to avoid DDOS (Distributed Denial of Service) attacks
        2. Example: 500 requests / s
    4. Quota
        1. A limit to the number of requests a developer can send to the API.
            1. It is used to avoid abuse by developers
                1. If a developer paid for usage, this ensures that other developers will not use the API without payment
    5. Policy
        1. It changes the API behavior through configuration
            1. Dozens of policies:
                1. Adding authentication
                2. Performance
                3. ...

Tiers and Pricing

1. 6 Tiers:
    1. Consumption
    2. Developer
        1. < $50 / month
    3. Basic
    4. Standard
    5. Premium
        1. Extremely expensive: ~ $ 2.8k / month
    6. Isolated
2. Main differences:
    1. Developer has no SLA
    2. Others range from 99.95% - 99.99%
    3. With **Consumption** tier, we pay for what we actually use
        1. No constant monthly payment as with other tiers
3. **Consumption** has no developer portal
    1. Developer portal is offered with other tiers
4. **Consumption** has auto-scale
    1. Not offered with other tiers
5. **Premium** and **Isolated** have VNet support
6. If we want API Management to be secure in a VNet and in-accessible to other external resources, we need to go with Premium and Isolated tiers

Constructing API Management and Building Backend App

1. Portal:
    1. Search: API Management
        1.
            - Create
        1. Resource group: apimgmt-rg
        2. Region: West Europe
        3. Resource name: course-api-mutthoju (unique)
        4. Organization name: hike-co
        5. Administrator email:
        6. Pricing tier: Developer (no SLA required)
            1. Shows SLAs of other tiers
        7. Review + Create
        8. Create
            1. Takes a lot of time
                1. In some cases it can take an hour

1. Azure will send an email if the deployment completes

2. Backend apps:
    1. New VM
        1. Search: VM
            1.
                - Add
                1. Resource group: apimgmt-rg
                2. Virtual machine name: weather-vm
                3. Region: West Europe
                4. Image: Ubuntu Server
                5. Size: Standard B1s
                6. Authentication: (SSH public key is the preferred approach)
                    1. Username: apiadmin
                    2. Password:
                    3. Confirm password
                7. Networking
                    1. Public IP: + Create new
                        1. Assignment: Static
                        2. OK
                8. Review + Create
                9. Create
    2. Duplicate the page:
        1. Download PuTTY
        2. Select installer compatible with OS
        3. Click the installer
        4. Install
    3. Go to resource
        1. Copy Public IP
    4. Open PuTTY:
        1. Paste public IP
        2. Open
        3. username: apiadmin
        4. password:
        5. Installation:
            1. Pull a Node.JS Web API from Github account (backend for API management)
            2. `sudo apt install git`
            3. `sudo apt update`
            4. `sudo apt install nodejs`
            5. `sudo git clone https://github.com/memilavi/weatherAPI.git`
            6. `cd weatherAPI`
                1. `ls`
            7. `sudo apt install npm`
            8. `npm start`
                1. Listens to port 8080
                    1. It is not open for external access
                        1. Networking:
                            1. SSH port is open only
                                1. Never leave the port open to any but limit it to a predefined IP
                            2. Add inbound port rule
                                1. Destination port ranges: 8080
                                2. Protocol: TCP
                                3. Action: Allow
                                4. Add
                            3. Refresh
                2. Open browser tab:
                    1. :8080/api/weather
                        1. Simple output

Building the Second Backend App

1. Portal:
    1. Search: Function
        1. Function App
            1.
                - Create
                1. Resource group: apimgmt-rg
                2. Function App name: hike-function-mutthoju
                3. Publish: Code
                4. Runtime stack: Node.js
                5. Region: West Europe
                6. Review + Create
                7. Create
            2. Go to resource
            3. Functions:
                1.
                    - Add
                    1. Development environment: Develop in portal (Other options: VS Code, Any editor + Core)
                        1. We put the code in the portal (VS code is not required)
                    2. Trigger: HTTP trigger
                    3. New Function: hikeinfo
                    4. Authorization level: Anonymous
                    5. Add
            4. Code + Test
                1. Copy and paste hike.js from resources
                    1. It gets the name of a hiking trail and returns information about it
                2. Save
                3. Test/Run

1. HTTP method: GET
2. Query:
    1. Name: name
    2. Value: Rainier
3. Run
    1. 200 OK

Adding APIs

1. course-api-mutthoju
    1. APIs
        1. Define a new API
        2. Function App
            1. Browse
                1. Configure required settings
                    1. hike-function
                    2. Select
                    3. Display name: hike info API
                    4. API URL suffix: hike/info
                    5. Create
            3. Two methods:
                1. GET hikeinfo
                    1. Click:
                        1. We can set up:
                            1. Frontend (we usually don't need this)
                            2. Inbound processing
                                1. We can define policies (affects the behavior of API call)
                            3. Backend
                            4. Outbound processing
                                1. We can define policies (affects the behavior of API call)
                        2. Click Test
                            1.
                                - Add parameter:
                            1. Name: name
                            2. Value: Renier
                            3. Reqeust URL: https://course-api.azure-api.net/hike/info/hikeInfo?name=Reinier
                                1. We are calling API management
                            4. HTTP request:
                                1. We are going to use subscription key
                            5. Copy the URL and paste in browser tab:
                                1. 401 - unauthorized
                                    1. Access is denied due to missing subscription key
                                        1. Go to function app:
                                            1. App keys:
                                                1. apim-course-api:
                                                    1. The key was created by API management when the API was created
                                                        1. Allows access from API management to function App
                                2. Click: Named values
                                    1. Key (automatically added)
                                        1. hike-function-key (same as the one in function app)
                            6. Hike info API
                                1. Settings:
                                    1. Subscription required: checked
                                        1. Header name: Ocp-Apim-Subscription-Key (header we saw when testing)
                                        2. Uncheck
                                    2. Save
                                2. Test
                                    1.
                                        - Add parameter
                                    1. name: name
                                    2. value: Renier
                                    2. Refresh the URL in the browser
                                        1. We have got the info
                2. POST hikeinfo
                    1. Remove it
    2. APIs
        1.
            - Add API
        1. Create from definition
            1. OpenAPI (Swagger formerly)
                1. OpenApi Specification: https://conferenceapi.azurewebsites.net/?format=json
                    1. Provided by Microsoft (demos adding OpenAPI)
                    2. Create
        2. GetSessions
            1. Test
                1. Send
                    1. Long JSON
                        1. Details of various sessions in the conference
            2. Settings:
                1. Uncheck subscription required
                2. Save
            3. Copy the URL and paste it in browser
                1. Same output
    3. Remove the API
        1. Click ...
            1. Delete

2. Yes
4. Weather API
    1.
        - Add APIs
    1. Blank API
        1. Display Name: Weather Info API
        2. Web service URL:
            1. http://:/api
        3. Create
        4.
            - Add operation
            1. Display name: Weather Info
            2. URL: /weather
            3. Description: Get an extremely accurate weather forecast
            4. Save
        5. Test
            1. Send
    2. Copy the URL and paste in browser
        1. Unauthorized
            1. Settings
                1. Uncheck subscription required
                2. Save
    3. Refresh the browser page

Using Products

1. Products: A logic container of APIs with some common settings such as terms of use etc.
2. API Management page:
    1. Products
        1.
            - Add
        1. Display name: Nature API
        2. Description: A collection of nature related APIs
        3. Published: No
        4. Required subscription: Yes
        5. Required approval: No (doesn't require the approval of administrator)
        6. APIs
            1.
                - (add APIs to be included in the product)
            1. Hike Info API
            2. Weather Info API
        7. Create
    2. Access control:
        1. Accessed only by Administrators
            1. Related to Developer portal
                1. We can use the Developer portal to define the access groups that can access the developer portal and use it
                2. Administrators group is the group with the maximum permissions to access the developer portal
    3. Click on: Nature API
        1. Summary of API
            1. APIs included
            2. Access control
            3. Not published yet
            4. Included in a single subscription
        2. Click: Access control
            1.
                - Add group
            1. Check: Developers, Guests
            2. Select
        2. All access groups are allowed to use the product
    4. Go to Overview:
        1. Click: Publish
            1. When using the API, the product's settings take effect
    5. Duplicate Portal:
        1. API Management:
            1. APIs
                1. Weather Info API
                    1. Select operation
                        1. Settings
                            1. Subscription is not required at the API level (But subscription is required at the product level)
                2. Test:
                    1. Copy the URL
                        1. Paste the URL in a browser
                            1. We can use the API
                                1. We need both the Product and API to require the subscription in order for the API Management to ask us for the subscription key
                                    1. Go back to the API and check **Subscription required**
                                        1. Save
                    2. Refresh the page
                        1. We need subscription key
                    3. Go to Product:
                        1. Settings:
                            1. Uncheck requires subscription and save
                        2. Go to browser and refresh the page
                            1. We can use the API
                                1. Solution: Both APIs and Products require the subscription

Subscriptions

1. So far the APIs we defined were completely open
    1. Anyone who had the URL can use them
        1. Not the desired case
            1. We want to limit to the developers who registered for the API
                1. Registration could be accompanied by some kind of payment
2. With subscriptions, developers subscribe to the API and get a subscription key which should be used when accessing the API
3. Subscription key ensures that the developer using the API is actually registered to the API and is actually allowed to do that
4. How does the registration occur?
    1. It can be done from the Portal
    2. It can be done from the developer portal
        1. Difference: How does the developer get the subscription key
            1. With portal, the admins have the subscription key and it will be provided to the developer
            2. With developer portal, the process is automated
                1. The developer can browse the APIs, locate the APIs that interest him, register for the APIs and get the subscription key
                    1. Admins have nothing to do with this process
5. Subscription Scope:
    1. Required to be defined when we define subscription
    2. Subscription can affect three types of resources:
        1. Subscription can be for
            1. A specific API
            2. All the APIs
            3. A specific Product

Using Subscriptions

1. course-api
    1. Subscriptions
        1. Some subscriptions were created automatically when constructing the API management
        2. They have a primary key and a secondary key
            1. Click ...
                1. Show/Hide keys
                    1. The keys must be used when accessing APIs that require subscription
            2. Ensure that both Product and APIs require subscription
                1. Check
                    1. Test using browser
2. Use PostMan
    1. Download and install
    2. Open PostMan (It is used to test REST APIs)
3. Go to APIs for Hike Info
    1. Operation
        1. Test
            1. Query parameters:
                1. Name: name
                2. Value: Yosemite
            2. Copy the URL
                1. Paste the URL in PostMan
                2. Click Send
                    1. Check subscription required to fix
                3. Click Send
                    1. 401 error message
4. Go to API page
    1. Click Test
        1. Select the operation
            1. HTTP request:
                1. Click the eye button to the subscription key
                    1. Where did it come from?
                        1. Subscriptions: Primary key
                            1. It is a built in all access subscription and provides access to all APIs in the API Management
                                1. We don't want to use this
5. Go to PostMan
    1. Copy the name of the header
        1. Paste it under header name
    2. Copy the subscription key
        1. Paste it under the header value
    3. Hit send
        1. We get the information
6. Own subscription:
    1. Subscriptions:
        1.
            - Add subscription
            1. hike-sub
            2. Display name: Hike Subscription
            3. Scope: API
            4. API
                1. Hike Info API
            5. User: Administrator
            6. Save
        2. Copy the primary key
    2. Go to PostMan
        1. Paste the primary key
        2. Send
            1. It works
                1. Modify last character of the key and send
                    1. It doesn't work

Policies

1. Policies change the API behavior using plug-and-play definitions
2. With policies we can change the inbound and outbound behavior
3. Policies are extremely useful for:
    1. Caching
    2. Validation
    3. Performance
    4. Changing values (in the request or response)
    5. ...
4. The policies can be heavily customized

Adding IP & Cache Policies

1. course-api
    1. APIs
        1. Hike Info API
            1. Inbound processing policies (base policy by default)
                1.
                    - Add policy
                    1. A limited list of inbound policies
                    2. Discard
                2. Adding policy using code (more flexible)
                    1. Policy code editor button

```
<policies>
    <inbound>
        <base />
        <!-- click Show snippets
            We can add to editor
            Dozens of them
            Select "Restrict caller's IPs - restricts access to the APIs for specific IPs"
        -->
        <ip-filter action="allow">
            <address-range from="10.1.1.1" to="10.1.1.2" />
        </ip-filter>
    </inbound>
    <backend>
        <base />
    </backend>
    <outbound>
        <base />
    </outbound>
    <on-error> <!-- policy triggered when an error occurs -->
        <base />
    </on-error>
</policies>
```

                    1. Save
                        1. We can see the policy in the editor
                    2. Testing:
                        1. PostMan
                            1. Copy the URL
                            2. Header must point to the right subscription key
                            3. Send
                                1. 403 - forbidden
                                    1. IP address is not in the allowed range of IP addresses
                    3. Delete the policy:
                        1. Click ...
                        2. Delete
                        3. Save
            2. Outbound processing policies (base policy by default)
                1. Complex policy: Cache
                    1. To improve response performance of the API
                    2. Outbound:
                        1. Code editor

```
<outbound>
    <base />
    <!-- show snippets
        Store to cache
    -->
    <cache-store duration="60" /> <!-- seconds - data is stored for 60 seconds when data is returned from API
</outbound>
```

                    3. Inbound:
                        1. When requesting from the API, we need to check whether the data is in the cache and not send request to the underlying API but return from the cache

```
<inbound>
    <base />
    <cache-lookup vary-by-developer="false"
    vary-by-developer-group="false"
    downstream-caching-type="none"
```

```
must-revalidate="true" ...>
    <!-- <vary-by-header>header name</vary-by-header> -->
    <vary-by-query-parameter>name</vary-by-query-parameter>
</inbound>
```

1. `vary-by-query-parameter` - store data in cache per query parameter
    1. We want to store different data for `Yosemite`, ...
        1. The name of the parameter is used to differentiate the data to be stored

4. Testing:
    1. Duplicate Portal page:
        1. Function App: hite-function
            1. Functions
                1. hikeInfo function
                    1. Code + Test
                        1. Difficulty of Yosemite hike is moderate
    2. Send the request in PostMan
        1. Difficulty is moderate
    3. Change the difficulty level to hard
        1. Send the request again
            1. Request returned much faster
            2. The difficulty level is still moderate (it didn't get updated)
        2. Cache works
    4. Send reqest with CaresRiver
        1. We will get data for CaresRiver
            1. Log records show that the actual API was invoked
    5. Send the request again
        1. Info is received much quicker
        2. Log records do not show that the function is invoked
    6. Send the request with Yosemite (more than 60s have passed)
        1. Difficulty level is hard

3. Backend policies (base policy by default)

Adding Rate Limit Policy

1. Rate limit limits the maximum concurrent reqeusts the API handles
2. Requests start coming to the API
    1. At some point, the load on the API is so big that the API crashes (not useful anymore, no other user can make requests)
    2. Solution: Rate limit
        1. A rate limiter is put in front of the API
        2. Requests first come to the rate limiter
        3. When there are too many requests to the rate limiter, it simply blocks the requests and returns a response code signaling that the server is too busy and cannot accept anymore requests
        4. API is up and running and fully functional
        5. Once API completes some of the requests, then API will receive new requests
3. course-api
    1. Inbound processing
        1. Code editor

```
<inbound>
    ...
    <!-- add Limit call rate per subscription -->
    <rate-limit-calls="3" renewal-period="10" remaining-calls-header-name="rate-limit-remaining">
```

            1. Maximum number of calls that are allowed per renewal period (for the timeframe)
                1. 3 calls / 10 seconds
                    1. More than that, API will return an error
            2. `remaining-calls-header-name` - Sets the name of the header that is returned to us that indicates of remaining calls we can run in the renewal period timerframe
            3. Click Save
4. Send request from Postman
    1. Headers:
        1. rate-limit-remaining: 2
            1. 2 calls remaining before rate limiter kicks in
    2. If we run multiple times, the policy didn't take effect
        1. Aspect: order of policies is extremely important
            1. When APIM entered **cache-lookup** and found the data in the cache, it returned the data
                1. APIM didn't check rate-limit policy further
            2. Solution: Drag the **rate-limit** policy before **cache-lookup** policy
                1. Code shows the order
    3. Send the request a few times
        1. 429 response code (too many requests)
5. We can almost always improve API management using policies

Full List of Policies

1. There are dozens of policies for API management
2. Full list of policies, their functionality, and how to use them
3. Policies are most important and most useful feature of API management
    1. Before we add some functionality to API, we can check if the functionality is already covered by existing policies

Securing the Backends

1. Currently the backends are publicly accessible

1. API managmenent accesses backends
    1. API management and backends are publicly accessible
        1. We can skip the API management and go straight to Hike API and Weather API
            1. The only way should be through the API management
2. How to do that?
    1. The way to achieve that depends on the backend type
        1. Examples:
            1. VMs - Put the API management in a VNet and set up peering with the VM's VNet
            2. Function - Block the function from access from outside
                1. Allow traffic to function only from API management

Deploying API Management in VNet

1. Portal:
    1. search: virtual networks
        1. We want to make sure that the address spaces of the new VNet and existing VNet do not overlap
            1. Go to apimgmt-rg-vnet
                1. Duplicate the tab
                    1. Go for VNet page
                        1.
                            - Create
                        1. Resource group: apimgmp-rg
                        2. Name: apimgmp-vnet
                        3. IP addresses
                            1. Address space (check if this overlaps with the address space of API M)
                        4. Review + Create
                        5. Create
            2. Search: Network security groups
                1. APIM requires NSG to limit the traffic (NSG - is like a mini firewall that restricts traffic based on source ip, port, and protocol)
                2. One NSG was created automatically for the VM of the weather api
                3.
                    - Create
                1. Resource group: apimgmt-rg
                2. Name: apimgmp-subnet-vnet (NSG is per subnet)
                3. Review + Create
                4. Create
            3. Go to the resource
                1. Inbound security rules
                    1.
                        - Add
                        1. Service: HTTPS (allows only HTTPS traffic through NSG)
                        2. Name: HTTPS
                        3. Add
                    2. A specific rule needs to be added for API Management (required for APIM to work with Azure resources)
                    3.
                        - Add
                        1. Source: Service Tag
                        2. Source service tag: ApiManagement
                        3. Destination: VirtualNetwork
                        4. Destination port range: 3443 (API Mgmt uses this port inside Azure)
                        5. Protocol: TCP
                        6. Priority: 110
                        7. Name: apimgmp-port-3443
                        8. Add
                    4. Assign NSG to subnet
                        1. Subnets
                            1.
                                - Associate
                            1. Virtual network: apimgmt-vnet
                            2. Subnet: default
                            3. OK
                        2. The subnet is going to have APIManagment
                    5. Public IP (will be used by APIM when placed in vnet)
                        1. Search: Public IP
                            1. Public IP Addresses
                                1.
                                    - Create
                                1. Name: apimgmt-ip
                                2. DNS name label: azapimgmtmutthoju (unique)
                                3. Resource group: apimgmt-rg
                                4. Location: West Europe
                                5. Create
            4. Go to API Management
                1. course-api
                    1. Deployment + Infrastructure
                        1. Virtual network
                            1. Takes a long time - changes can take from 15 to 45 minutes to apply
                            2. Select either **external** or **internal** VNet (external - APIM will be accessible from the internet, internal - APIM will be accessible only to internal Azure resources)
                            3. Select **external** (APIM is used from the internet)
                                1. Location: (APIM can be deployed into VNets that are located in the same location as the APIM)
                                    1. Select West Europe
                                        1. Virtual network: apimgmt-vnet
                                        2. Subnet: default
                                        3. Public IP address: apimgmt-ip
                                        4. Apply

2. Save

Securing the Weather and Hike Backends

1. Azure Functions: Three types of protection can be used
    1. Limit traffic to only Azure services (only Azure services can access the function)
    2. Allow traffic only from the newly created subnet
        1. Only traffic originating from the subnet is allowed
    3. Using identity to authenticate the request arriving to the function
2. Search: Function App
    1. hike-function
        1. Settings
            1. Networking
                1. Access Restrictions
                    1. Configure Access Restrictions
                        1. We have only default rule (all traffic)
                        2.
                            - Add rule
                        1. Name: Allow API Management
                        2. Priority: 100
                        3. Action: Allow
                        4. Type: Service Tag (first option) (definition of various services within Azure and outside Azure which represents the IP addresses of the service - we don't have to know the IP address)
                        5. Service Tag: ApiManagement
                            1. Second option:
                                1. Type: Virtual Network (traffic originating from a specific VNet is allowed to the function app)
                                2. Subscription: Azure subscription 1
                                3. Virtual Network: apimgmt-vnet
                                4. Subnet: default
                                    1. Message: The default subnet doesn't have a service endpoint enabled for Microsoft.Web (it is the namespace of the function app). Enabling access may take upto 15 minutes to complete
                                        1. If we enable this rule, Azure will add a service endpoint to the default subnet to allow access to the function app
                                            1. Service endpoint is two sided action
                                                1. First we need to allow traffic to underlying resource (function app)
                                                2. We need to define service endpoint on the originating subnet (default subnet)
                                5. Add rule
                                    1. Azure adds service endpoint
                                        1. Search: Virtual Networks
                                            1. Click: apimgmt-vnet
                                                1. Sbnets
                                                    1. default
                                                        1. Services: Microsoft.Web
                                                            1. Allows traffic from VNet to the function app resource we set up
3. Protecting VM so that only traffic from APIM will be allowed
    1. Search: Virtual Machines
        1. weather-vm
            1. Click the VNet
                1. Peering (process of connecting two VNets so that resources in one VNet can connect to resources in the other VNet)
                    1. Addresss spaces between the VNets should not overlap
                    2.
                        - Add
                    1. Peering link name: weather-apimgmt-peer
                    2. Remote link name: apimgmt-weather-peer (from APIM)
                    3. Virtual network (VNet to peer with)
                        1. apimgmt-vnet
                    4. Add
                        1. Two peerings are defined one the the other and back
4. Function app:
    1. hike-function
        1. hikeInfo (function)
            1. Get function url

                1. Copy and paste in new browser tab

                ```
                <url>?name=Renier
                ```

                1. 403 - forbidden
                    1. The rule is working

Connecting API Management to Secure Backends

1. APIM is inside a VNet
    1. Network connectivity status
        1. All green
            1. APIM can reach all the resources it needs and all backends
2. Run the same request in PostMan
    1. Hike info
        1. We have got the info
            1. APIM can reach the backend
    2. Run it a few more times to check if rate limiting works
3. For VM
    1. Access from APIM to VM through private IP and not through public IP address (we want to remove the public IP)
    2. weather-vm
        1. Copy private IP (we peered APIM VNet with VM VNet so APIM must be able to use private IP to access the VM)

4. Go to APIM
    1. course-api
        1. APIs
            1. Weather Info API
                1. Settings
                    1. Paste private IP address
                        1. Web service URL: http://:8080/api
                2. Test
                    1. Select the operation
                        1. Send
                            1. APIM was able to access VM
5. Go to weather-vm
    1. Networking
        1. Delete the rule Port_8080
6. Go back to test
    1. Hit send
        1. We got the response

Other Security Options

1. There are more ways of securing the backend
    1. Protect Backend Web API using OAuth 2.0 and Azure AD:
        1. https://docs.microsoft.com/en-us/azure/api-management/api-management-howto-protect-backend-with-aad
    2. Adding Application Gateway in front of API Management:
        1. https://docs.microsoft.com/en-us/azure/api-management/api-management-howto-integrate-internal-vnet-appgateway
    3. Use certificates with API Management
        1. https://docs.microsoft.com/en-us/azure/api-management/api-management-howto-ca-certificates

Versions and Revisions

1. Over time, APIs are going to change
    1. The changes must be well managed in order to not break things that the developers use
2. What are versions?
    1. Version: A group of related API for the developers
    2. Each version represents a specific set of capabilities
    3. New versions usually break the old onces
        1. If the developers switch from older versions to newer ones, the code will work differently
        2. One of the roles of versions is to announce that a breaking change is part of the system
    4. Developers can indicate the desired verison using:
        1. Path (required version in the path)
        2. Query String
        3. Header
    5. Examples:

        ```
        api/cars/v1/models
        ```

        ```
        api/cars/fastest?v=2
        ```

    6. Versions are for the API users (=developers)
3. API Revisions:
    1. Revision = A non-breaking change to the API definition
        1. Example: Changes in policies
            1. Usually done in new revision (no new version is released)
    2. Revision allows making the changes safely without affecting the published API
    3. Revisions can be tested and documented before publishing
        1. Only after we are satisfied with the result, we can publish the revision and make it the current version, the API users will use
    4. Revisions can be rolled back in case of problems
        1. If we find out that the new policy definition is not good, we can roll back to the original version
    5. Examples:
        1. Adding rate limit policy
        2. Adding validation policy (for parameters)
    6. Revisions are for the API developers (not the users)

Using Versions

1. course-api
    1. APIs
        1. Hike Info API
            1. ...
                1. Add version
                    1. Add new version to the Hike Info API
                    2. Version identifier: v1
                    3. Versioning scheme: Path (part of the path of the API)
                    4. Full API version name: hike-info-v1
                    5. Products: Nature API
                    6. Create
            2. Two versions exist:
                1. Original: uses the original URL we defined previously
                2. v1: has its own URL

3. Click Hike Info API
    1. Version set: It is a logic container of versions for an API (we can define verious properties for the version set)
        1. Display name
        2. Description (what are the versions are for)
4. Click: v1
    1. Test:
        1. Click the operation
            1. Request URL: https://.../v1/...
        2. Paste the URL in PostMan
            1. ?name=Ranier
                1. Error: We don't have subscription key
                    1. Subscriptions
                        1. Scope: API without any version
                            1. v1 is not included
                        2. Scope: Product Nature API
                            1. Show/Hide keys
                                1. Copy: Primary key
            2. Paste in PostMan
                1. Send
                    1. It works
                        1. The API is part of a product (it automatically includes any new versions and revisions of an API)
                            1. Subscription automatically includes the new version
                                1. One reason to use products instead of the APIs with subscription
    2. Portal:
        1. hike-function
            1. Functions
                1.
                    - Add
                1. Development environment: Develop in portal
                2. HTTP trigger
                3. New Function: hikeinfoview1
                4. Authorization: Anonymous
                5. Add
            2. Code + Test
            3. Download hikeview1.js
            4. Copy paste its contents in the function body
            5. Code:
                1. For kid's is it easy enough to do the hike
            6. Save
    3. APIM
        1. APIs
            1. v1
                1. hikeinfo operation
                    1. Inbound policy
                        1.
                            - Add policy
                        1. Rewrite URL
                            1. Backend: /hikeinfoview1
                2. Test
                    1.
                        - Add parameter
                        1. name = Renier
                        2. forKidsIndicator = true
                    2. Send
                    3. Send request again
                    4. Change indicator to false and send again
                        1. yes indicator
                            1. Why?
                                1. Cache policy - only name parameter is used
                                    1. Design
                                        1. cache-lookup

```
<vary-by-query-parameter>forKidsIndicator</vary-by-query-parameter>
```

                    5. Send request again
                        1. Modification works
                    6. Call the original version from PostMan
                        1. It still works like before

Using Revisions

1. course-api
    1. APIs
        1. v1
            1. Revisions
                1.
                    - Add revision
                1. Revision description: Increase rate limit to 10 (what we want to achieve through this revision)
                2. Create
            2. The revision is online but not the current revision

1. Current revision is the previous one
3. Title: Revision 2 is what we are working on
    1. We can select Revision 1 and working on it
4. Design
    1. rate-limit policy

        ```
        <rate-limit calls="10" ...>
        ```

    2. Save
5. Go to Revision 1
    1. rate-limit policy
        1. It is still 3

2. Revisions
    1. /hikeInfo?rev=2
        1. Used to test the new revision before publishing it
    2. Test:
        1. Send request in PostMan using `rev=2`
    3. ...
        1. Make current
            1. Post public change log for this API
                1. Log that documents the changes that happened to the API between versions and revisions
                2. Type: Rate limited increased to 10 due to infrastructure improvements
                3. Save
            2. Revision 2 is the current one
        2. Create version from this revision
            1. If the new revision is a big one and probably has breaking changes
    4. Change log
        1. Documents the change we made
        2. Test: Remove `rev=2` and send the request in PostMan
            1. rate limit increased

Monitoring

1. It's important to know the state of the API managment
    1. Whether there are problems, is everything fine, or is there anything that needs our attention
        1. Solution: Monitoring is an important part to figure out what is going on with the API management
    2. Monitoring in API management uses the regular monitoring and logging mechanism of Azure
        1. Metrics
        2. Logs
        3. Application Insights
    3. Some more mechanisms exist
        1. We'll focus on:
            1. Analytics
            2. Metrics

Using Monitoring

1. course-api
    1. Monitoring
        1. Metrics
            1. Metric: Capacity
                1. The capacity used by API management (compute power used in 24 hours default)
                    1. If it is too high, we need to scale up the API Management and select a stronger tier
            2. Metric: Requests
                1. The number of requests handled by API management in 24 hours
                2. Change to last 7 days
                3. One of the most important
                    1. New alert rule
                        1. Scope: course-api
                        2. Condition: whenever the sum of requests greater than
                        3. Actions
                            1. Action group
                        4. Alert rule details
        2. Analytics
            1. We get information about API Management behavior and usage
            2. Time range: last 7 days
                1. Requests
                2. Data transfer
                3. Response time
                4. Cache (usage)
            3. Geography
                1. Heatmap
                    1. Where did the requests came from
                        1. For each country
            4. APIs
                1. What actual APIs were called and how many successful requests, unauthorized, ... were received by the APIM
            5. Operations
                1. Specific URL inside an API
            6. Products
                1. Products usage of APIM
                    1. Nature API - only usage
            7. Subscriptions
                1. Subscriptions used
                    1. Actual usage per subscription

8. Users (number of requests per user)
    1. Anonymous
    2. Administrator
9. Requests
    1. List of actual requests that were handled by APIM
        1. To grill down into actual request and what happened

Developer Portal

1. A quick reminder:
    1. API Gateway - handles API requests and executes the policies
    2. Azure Portal - to define APIM, set up policies, set up APIs
    3. Developer Portal
2. Oftentimes, we want to advertise the API exposed via the API management
    1. So that developers can learn about our APIs and gain access to them (so that instead of developers sending emails asking for access keys)
        1. Doesn't need any human intervention
            1. Solution: Developer portal
                1. Shows APIs
                2. Allows testing
                3. Allows developer to register for the API and get subscription key
3. Developer Portal
    1. It is automatically generated, fully customizable web portal with docs about the API
    2. It includes interactive console to test the APIs
    3. It can be used to request access to the API
    4. It supports authentication via various identity providers

Using the Developer Portal

1. course-api
    1. Developer portal URL
        1. Copy
            1. Paste the link in in-cognito mode browser tab (for anonymous user)
                1. Developer portal is not published yet
                    1. Portal:
                        1. Portal overview
                            1. Enable CORS (allows cross domain access between websites)
                            2. Click Developer Portal (as admin)
                                1. Editor
                                    1. Logo
                                    2. Descriptions
                                    3. Click Welcome to Contosos!
                                        1. Welcome to Hike Co.!
                                    4. Click logo
                                        1. Click recycle bin
                                    5. Testing portal for various form-factors
                                        1. Phone screen (for developers using mobile phone)
                                        2. Wide screen
                                    6. Save
                                3. Click on side-bar
                                    1. Operations
                                        1. Publish Website
                        2. We can publish from Azure Portal as well
                        3. Copy the link
                        4. Paste the link in-cognito window
                        5. Click APIs
                            1. Two versions of Hiki Info API
                            2. Click Hike Info API
                                1. Operation
                                    1. We can change version
                                    2. We can look at change log
                                    3. Change to v1
                                    4. Change log shows adding revision
                                    5. Click try it
                                        1. We need subscription
                                            1. We need to register to the developer portal
                                    6. Sign Up
                                        1. Fill in details
                                        2. Sign Up
                                    7. Copy the link sent to email
                                        1. Paste the link in-cognito
                                        2. Sign in
                                            1. Products
                                                1. Nature API
                                                    1. Nature API Subscription
                                                        1. Subscribe
                                                2. Hike Info API
                                                    1. try it
                                                        1. Subscription key is pre-populated
                                                        2. Add parameter
                                                            1. name: Reinier
                                                            2. Send
2. We can select the language for the API
    1. C#
        1. Code to use to use the API
    2. Java
    3. Javascript

4. PHP
5. ...

Adding Azure AD Support for the Developer Portal

1. Identity provider is basic one by default
2. Using AD Authentication
    1. Portal
        1. course-api
            1. Portal overview
                1. Date published is shown in Overview
                2. Click Publish here to make changes get reflected in Developer Portal
                3. Enable user sign-in with Azure Active Directory
                    1. Enable Azure AD
                    2. Enable Azure AD
                        1. Azure registers the developer portal as an application in the Azure AD
                            1. A token is created
                            2. Adds it as an identity provider in APIM
            2. Identities
                1. Two providers
                    1. Azure Active Directory
                    2. Username and password
                2.
                    - Add
                    1. Type:
                        1. Facebook, Google, Microsoft, Twitter (deprecated - because this functionality is supported using Azure Active Directory B2C)
                        2. Azure Active Directory B2C
            3. Portal overview
                1. Publish
                    1. AD is configured
            4. Open URL in-cognito
                1. Click Sign in
                    1. Azure Active Directory button (enables users in AD to login)
3. Summary of APIM:
    1. Allows us to mask internal APIs
    2. Protect the internal APIs
    3. Change their behavior
    4. Expose a consistent and easy to use APIs
    5. APIM is used if there are many APIs in the org
4. Remove the resource group of APIM

Resources

1. To learn more:
    1. API Gateway can be local (on-prem)
        1. It can be managed using Azure portal:
            1. https://docs.microsoft.com/en-us/azure/api-management/self-hosted-gateway-overview
    2. Modifications to developer portal (including code):
        1. https://docs.microsoft.com/en-us/azure/api-management/api-management-howto-developer-portal-customize
    3. We can add API facades before the underlying backends even exist
        1. We can use the mocking capabilities of API Management to mock response of an API that does not have a backend yet:
            1. https://docs.microsoft.com/en-us/azure/api-management/mock-api-responses?tabs=azure-portal
    4. Tracing: To find out what happened to the request sent to API Management
        1. Tracing capability of APIM provides rich info about the request (useful in figuring out bits and bytes of request)
            1. https://docs.microsoft.com/en-us/azure/api-management/api-management-howto-api-inspector

Section 6: Notification Hub

Introduction

1. What is it?
    1. It is a central hub for sending push notifications to mobile devices
        1. To user's devices
    2. It is massively scalable
        1. Supports millions of notifications and devices
    3. It is cross-platform:
        1. Android
        2. iOS
        3. Windows
        4. Amazon
2. It can be used from almost any backend
    1. Cloud or on-promise
        1. We don't have deploy app cloud
3. It is extremely cost effective
4. It uses tags to target specific devices
    1. Sometimes we want to segment users and send notifications to a subset of the users

Push Notification Concepts

1. Main concept:
    1. PNS
        1. Platform Notification Systems
            1. It is a platform for pushing notification to device
            2. It uses device's handle (a unique token a device uses to register itself in a PNS)

3. It is platform specific
2. How PNS works:
    1. First device registers with device handle in PNS (PNS becomes familiar with the device and knows how to communicate with it)
    2. App (backend) sends notification request to PNS (it doesn't communicate with the device directly)
    3. PNS sends notification to the device
        1. PNS is platform specific
            1. Flow is duplicated and is specific per platform
                1. APNS - Apple PNS
                2. FCM - Firebase Cloud Messaging (Google)
                3. WNS - Windows Notification Service
            2. App Needs to communicate with each one of the PNSs
                1. Problem:
                    1. App needs to employ separate APIs
                    2. Separate handles (each PNS has its own handle format)
                    3. Limited functionality - the app needs to adapt itself to the various functionality of PNS and to work with common denominator of all the PNSs
                2. Solution:
                    1. Notification Hub
                        1. Azure NH sits in between the app and PNSs
                            1. App only needs to connect with the NH and send notification
                                1. NH does the dirty work of connecting with all the PNSs and communicating with them and ensuring that everything works flawlessly regardless of specific PNS and specific device a user has

Tiers and Pricing

1. Three tiers:
    1. Free
        1. For 1 million
    2. Basic
        1. $10 / month for first 10 million notifications
    3. Standard
        1. $200 / month for first 10 million notifications
2. Main differences:
    1. Free has no SLA, Basic and Standard have 99.9% SLA
        1. Don't use Free tier for Production (especially with heavy load)
        2. Good for Dev and Test scenarios
    2. Free inludes 1 m notifications / month, Basic and Standard include 10m free notifications
    3. Devices / namespace: Free = 500, Basic = 200K, Standard = 10m
        1. Number of devices

Using Notification Hub

1. Portal:
    1. Search: Notification
        1. Notification Hubs
            1.
                - Create
            1. Resource group: new
                1. notifhub-rg
            2. Namespace: notifhub-nf (container to notification hub)
            3. Notification hub: notifhub
            4. Location: West Europe
            5. Select pricing tier: Free
            6. Create
2. Android app:
    1. Download and install Android studio
        1. Download Android studio
        2. Install
    2. Start Android studio
        1. Do not import settings
        2. ...
        3. Standard
        4. ...
        5. Finish
3. Download notificationhubandroid.zip file
4. Extract it
5. Launch Android studio
    1. Open an existing project
        1. Open the folder where the files are
6. Firebase cloud messaging account (PNS used by Google to send notifications to Android mobile phone)
7. Go to: console.firebase.google.com
    1. Sign in or sign up
    2.
        - Add project
        1. Name: fcmnotificationhub
        2. Allow google analytics
        3. Default google analytics account (created for Firebase)
        4. Create project
        5. Continue
    3. Add Firebase libraries and support for our App (Android)
        1. Click Android icon
            1. Register app:
                1. Package name: com.example.mobilenotificationtest
                2. Register app
            2. Download config file (google_services.json - needs to be merged into our project)

1. Download
2. Open the folder the file is in
    1. `app` folder
        1. Paste the file

4. Firebase
    1. Settings button (near Project Overview)
        1. Project settings
            1. Cloud Messaging
                1. Copy Server key (server key used when sending messaging to Android phones when using Firebase cloud messaging)
5. Azure Portal:
    1. Go to the resource
        1. Max devices - 500
        2. Free pushes - 1 M
        3. Manage
            1. Access Policies
                1. DefaultListenSharedAccessSignature (connection string allows us to listen to notifications sent from the notification hub)
                    1. Client application (mobile app) should use this policy
                        1. Azure displays a message to do this
                2. DefaultFullSharedAccessSignature (to listen, manage notification hub, send notifications)
                    1. We can use this to connect to notification hub from backend code and to send notification to devices
            2. Google
                1. API Key
                    1. Paste the API Key we copied from Firebase cloud messaging page
                    2. Save
            3. Access Policies
                1. Copy DefaultListenSharedAccessSignature
                2. Android studio
                    1. MainActivity.kt
                        1. Paste after `connectionString`
                    2. Open NotificationHubListener.java
                        1. Set break point at line 18 (`onPushNotificationReceived` method)
                            1. Triggered when a new push notification is recieved by the app
                    3. Shift + F9 (build)
                        1. Phone is opened (emulator)
                            1. Android app is up and running
            4. Portal
                1. Overview
                    1. Test Send
                        1. Platforms: Android
                        2. Send
                            1. Sent successfully
                            2. Breakpoint was hit
                                1. F8 (to advance)
                                2. `title` variable (title of the message)
                                3. F8
                                4. `body`
                                5. F8
                                6. Writing content to log
                                7. Data
                                8. Click Logcat (bottom left)
                                    1. key/value pairs
                                9. Ensure Emulator that is running is picked
                                10. Ensure that Show only selected application is selected
                2. Refresh
                    1. Some successful push notifications
                    2. How many push notifications were sent

Tags and Tag Expressions

1. Not always we want to send push notifications to all devices (called broadcast)
2. Usually there's a need to send push notifications just to devices that belong to specific groups
3. Examples:
    1. Users interested in technology news
    2. Participants in specific chat
    3. Followers of specific influencer
    4. Specific user
4. For that we have the tags
5. Tags
    1. Tags are used for sending notifications to subset of devices based on properties of the device
    2. Devices can register specific tags and Notification Hub sends notifications to devices having these tags
    3. Devices can register one or more tags (up to 60)
    4. Tags are simple strings
    5. Examples:
        1. Tech_news
        2. Kim-followers
        3. User_558392
        4. Chat-882dkc
    6. Flow of Tags:
        1. Step 1: A device registers for a tag (Beatles) with Notification Hub
        2. Step 2: App backend sends notification to Notification hub targetting all devices with tag Beatles
        3. Step 3: Notification hub sends the notification to only those devices
    7. Tag Expressions
        1. Notifications can be sent to devices that meet tag conditions (not specific tags only)
        2. Example:
            1. Interested in tech_news and also follows Satya Nadella
            2. Users who belong to petrolHeads group but is not to specific user named 55838

3. Tag expressions can be used for the above scenarios
4. With Tag Expressions we can define logic expressions that determine whether a notification will be sent to the device
5. It uses common logical operators
6. Examples:
    1. `(tech_news || business_news) && location_France`
    2. `(group_5593 && !user_35828)`
7. Tag expressions add another logical layer to simple tags and allows further segmentation

Using Tags and Tag Expressions

1. In MainActivity.kt

```
onCreate(...) {
    ...
    NotificationHub.addTag("tech_news");
```

    1. Device is going to register tech_news tag with notification hub so that future notifications that will be sent to the tech_news tag will be recieved by the device

2. Debug the app

3. Portal:

    1. Test Send
        1. Platforms: Android
        2. Send
    2. Breakpoint is hit
    3. Test Send

        1. Send to Tag Expression: tech_news

        2. Payload:

```
"title": "Tech News Breaking News",
"Body": "This is an extremely important tech news!"
```

        3. Send

        4. Breakpoint is hit again

    4. Test Send
        1. Send to Tag Expression: fashion_news
        2. Send
        3. Breakpoint is not hit
    5. Test Send

        1. Send to Tag Expression: (tech_news || fashion_news)

        2. Payload:

```
"title": "Fashion of Tech News"
```

        3. Send

        4. Breakpoint will be hit

Sending Notifications Using Code

1. Sending notifications from our own code
2. Download the Notificationhubsample.zip file
3. Extract it
4. Go into the folder
5. Open the backend project in VS Code
    1. Program.cs
        1. NotificationHub client (a client for the Notification hub)
        2. Client is going to connect to the Notification hub using the connection string
        3. Commented lines: another way of sending notifications
            1. Construct notification we want to send
                1. Title
                2. Body
            2. `SendFcmNativeNotificationAsync` - A method to send notifications to Android devices
        4. Connection string:
            1. Portal:
                1. Access Policies
                    1. Copy DefaultFullSharedAccessSignature
                    2. Paste in `CreateClientFromConnectionString`
                    3. Save
        5. VS Code
            1. F5
            2. Log entry: Enqueued
                1. Notification was sent to notification hub
        6. Android studio
            1. Hit breakpoint
        7. VS Code
            1. `hubClient` (instance of Notification hub client)
            2. Sends notification to FCM (Firebase Cloud Messaging)
                1. There are other methods

        1. Check resources (one-stop-shop to working with Notificaiton Hub)
    6. Delete resource group

Resources

1. Notification hub has a lot of capabilities and features
2. Resources to learn more:
    1. `NotificationHubClient` class has a lot of useful methods for communicating with the Notification Hub
        1. One-stop-shop for all notification hub-related tasks
            1. https://docs.microsoft.com/en-us/dotnet/api/microsoft.azure.notificationhubs.notificationhubclient?view=azure-dotnet
    2. Notifications can be scheduled:
        1. https://docs.microsoft.com/en-us/azure/notification-hubs/notification-hubs-send-push-notifications-scheduled
    3. Templates can be used for a unified messaging format across all devices
        1. https://docs.microsoft.com/en-us/azure/notification-hubs/notification-hubs-templates-cross-platform-push-messages

Section 7: Cognitive Services

Introduction

1. What are they?
    1. A set of APIs to help us build cognitive intelligence apps
        1. The APIs use Artificial Intelligence, Machine Learning, and more to perform some cognitive actions
2. We don't need to know anything about AI and ML
    1. We just need to know how to use the APIs
3. It is not a regular service...
    1. We just get an API
4. Categories:
    1. Vision:
        1. Computer Vision
        2. Face recognition
        3. Form recognition
        4. ...
    2. Speech
        1. Speech-to-Text
        2. Text-to-Speech
        3. Translation
        4. ...
    3. Languages
        1. Language understanding
        2. QnA maker
        3. Text analysis
        4. ...
    4. Decision
        1. Anomaly detector
        2. Content moderator
        3. Personalizer
        4. ...
    5. Search
        1. Full text search
        2. Indexing
        3. ...

Pricing

1. Pricing structure is a little complicated
    1. But we basically pay for the number of transactions
2. Computer Vision:
    1. Face recognition -
        1. For First million transactions:
            1. $1 / 1000 transactions
        2. For 1 milltion to 10 million transactions
            1. $0.65 / 1000 transactions
        3. ...
3. Language
    1. We are not paying because the service exists
        1. We pay for the actual usage - similar to Azure functions

Constructing Cognitive Services

1. Portal:
    1. Search: Cognitive Services
        1. Two places
            1. Services
                1. Cognitive Services - routes to Marketplace
            2. Marketplace
                1. Cognitive Services
                    1. Resource group: Create one
                        1. cognitive-rg
                    2. Region: West Europe
                    3. Name: azcourse-cognitive
                    4. Pricing tier: Standard S0 (only one is avaialable)
                    5. Check to acknowledge that we have read and understood the terms
                        1. Responsible AI notice - what the cognitive services are intended for
                    6. Review + Create
                    7. Create

2. Go to resource
    1. Quick start guide
    2. Overview
        1. Status: Active
    3. Keys and Endpoint
        1. Used to connect to the API and work with it
        2. Copy the Endpoint

            1. Open Postman:

                1. POST: /test/analytics/v3.0/sentiment
                2. Headers:
                    1. Content-Type: application/json
                3. Ocp-Apim-Subscription-Key: Copy and paste Key 1
                4. Body:

                    1. Check Raw:

                    2. Download and copy paste resource text

```
{
    documents: [
        {
            "language": "en",
            "id": "1",
            "text": "I love my wife"
        },
        {
            "language": "en",
            "id": "2",
            "text": "My cat annoys me"
        }
    ]
}
```

            2. Sentiment API:

                1. It takes some text, analyzes, returns a sentiment (mood)
                    1. Are we angry
                    2. Are we happy
                    3. ...

            3. Hit Send

                1. Sentiment: Positive
                2. Sentiment: Negative

            4. Download HE file and paste contents of the file under the previous body

                1. Error: Invalid language code
                    1. List of supported languages

Langauge Understanding

1. Language Understanding Service (LUIS)
    1. It analysis texts and understands them
    2. Great for:
        1. Bots (hotel reservations, medical advice etc.)
            1. The bots usually need to understand what we are telling them so that they will know how to reply accordingly
        2. Voice controlled devices (home automation, car controls, etc.)
            1. We can talk to the home or car and they do what we want
                1. They need to understand what we exactly want from them
        3. ...
    3. Extracts **Intents** and **Entities**
        1. From the text that we provide
2. LUIS
    1. Intent
        1. It indicates **what we want to do**
            1. Examples:
                1. Turn off something
                2. Buy something
                3. Go somewhere
                4. Stand up somewhere
    2. Entity
        1. The object you refer to
            1. Examples:
                1. Light
                2. Cake
                3. Outside
                4. Chair
    3. By extracting the intent and entity, LUIS can exactly tell what we want to do
    4. In order to use LUIS, Intents and Entities should be defined
        1. Solution:
            1. There are pre-built domains for specific areas (saves us the need to manually configure the intent and entities)
                1. **Can be configured via the LUIS portal**
    5. No coding is required
    6. No ML knowledge required

Using LUIS

1. luis.ai
    1. Home page of language understanding service
    2. Login
        1. New authoring resource (resource used for LUIS capabilities)
        2. Pick subscription
            1. Creat an authoring resource
                1. Azure resource group: cognitive-rg
                2. Azure resource name: azcource-luis
                3. Location: West Europe
                4. Done
            2.
                - New app
                1. Name: home-app
                2. Culture: English
                3. Done
        3. Prebuilt Domains
            1. Calendar
            2. Communication
            3. Email
            4. NomeAutomation (select)
                1. Add domain
            5. Note
            6. Places
            7. RestaurantReservation
            8. ToDo
            9. Utilities
            10. Weather
            11. Web
        4. Click: Intent
            1. QueryState
            2. SetDevice (state)
            3. TurnDown
            4. TurnOff
            5. TurnOn
            6. TurnDown
        5. Click: Entities
            1. Device name
            2. Device type
            3. Location
            4. Numerical change (that can happen as a result of the action)
            5. ...
                1. We can customize, modify, and add
            6.
                - Create (for new entities)
        6. Training:
            1. Train - disabled
                1. Refresh the page
                    1. LUIS needs to figure out that we have Intents and Entities
                2. Click Train
                    1. LUIS will use ML capabilities to understand Intents and Entities the user will provide
        7. Manage
            1. We are going to add a prediction resource
                1. Prediction resource - a publicly accessible API which we can use to feed LUIS with text to be analysed
            2. Click: Azure Resources
                1. Add prediction resource
                    1. Prediction resource: azcourse-cognitive
                2. Done
        8. Publish
            1. Production Slot (API will be in production)
                1. Done
        9. Copy the example query
            1. It has all the parameters we want to use
                1. GET:
                    1. YOUR_QUERY_HERE -> please turn off the lights
                    2. Headers:
                        1. Uncheck: Ocp-Apim-Subscription-Key (it may confuse LUIS)
                    3. Send
                        1. Response:
                            1. Intent: HomeAutomationDomain.TurnOff
                                1. Highest rank: 99% probability
                            2. Entity: Lights
                        2. Turn off the Lights
                            1. Backend developer exactly knows what to do
                    4. Change to: Turn the heat up to 25 degrees
                        1. LUIS understands
                            1. Probability is > 98%
                            2. Entities:
                                1. SettingType: Heat
                                2. Setting: 25
                                3. Unit: Degrees
                        2. Using the above data, developer knows exactly what needs to be done
    3. LUIS can take free text, extracts intents and entities and gives the developer a structured data that contains everything the developer needs to know as to what exactly needs to be done.

Text to Speech

1. What is it?
    1. It converts written text to a synthesized speech
        1. Use-cases:
            1. Great for audio books
            2. Talking devices
            3. As an assistive feature
            4. (anything that requires speech)
2. It uses multitude of voices with various features
    1. We can provide the best voice for the text
    2. It can translate the text while converting it to speech
3. It can be implemented using:
    1. REST API
    2. SDK
        1. Downlaoded from cognitive services page
        2. Simple to use
    3. No need for special coding skills or AI skills

Using Text to Speech

1. Portal
    1. Search: Speech
        1. Marketplace: Speech (New text-to-speech resource)
            1. Name: azcourse-speech
            2. Subscription
            3. Location: West Europe
            4. Pricing tier: Free
            5. Resource group: cognitive-rg
            6. Create
        2. Go to resource
    2. API type: Speech
    3. Pricing tier
        1. Differences between various tiers of speech cognitive service
            1. Speech Transaltion:
                1. Free: 5 hours/month
                2. Standard: Unlimited but we pay
            2. Speech-to-text:
                1. Free: 5 hours/month
                2. Standard: Unlimited but we pay
            3. Text-to-speech:
                1. Free: 5 M characters/month
                2. Standard: Unlimited but we pay
    4. Keys and Endpoint
        1. How we are going to connect to the service and how we are going to use the API
        2. Open Postman:
            1. Copy and paste the Endpoint URL: https://westeurope.tts.speech.microsoft.com/cognitiveservices/voices/list
            2. Method: GET
            3. Check: Ocp-Apim-Subscription-Key:
            4. What does it do?
                1. Shows us various voices that can be used with the service
                    1. Send
                        1. Long list
                            1. For each voice we have some characteristics:
                                1. Name: "Microsoft Server Speech Test to Speech Voice (locale, Name of speeker)"
                                2. DisplayName: "Jacob"
                                3. Gender
                                4. Locale
                                5. Sample rate Hertz: Quality of sound
                                6. Status: GA
                                    1. Generally available
                                        1. Tested and it can be used
                                    2. Status: Preview
                                        1. It is not fully tested yet
                                            1. There might be some problems
                        2. Copy the list and paste it into notepad
                2. Change the request
                    1. Replace `voices/list` with `v1`
                    2. `POST`
                    3. Headers:
                        1. `Content-Type`: `application/ssml+xml`
                        2. `x-Microsoft-OutputFormat`: `audio-48khz-192kbitrate-mono-mp3` - for audio format (there are other formats)
                    4. Body:
                        1. Download the file and copy paste the content

                            1. XML document

                            ```
                            <speak ...>
                                <voice ...> <!-- specifies the voice we need - shortname for the list is used -->
                                    text
                                </voice>
                            </speak>
                            ```

                            1. Stylelist item - what are the best scenarios for the voice
                                1. Chat application - Aria is the best fit

                            2. SSML:

                                1. Speech Synthesis Markup Language

1. A flavor of XML - defines speech
5. Send
    1. Save response
    2. Save to a file
    3. Pick the folder where we want the file to be saved
    4. suffix - mp3
6. Double click the file
7. Another voice
    1. name: ""
    2. Send
    3. Save the response
        1. Translation is also possible

Computer Vision

1. The computer vision API analyzes images and extracts meaningful information from them
2. It can be used with SDKs in various languages or REST API
3. Usage of SDK and API is extremely simple
4. No AI or ML knowledge is required
    1. We just need to know how to use the SDK or API
5. APIs:
    1. OCR - Optical Character Recognition
        1. It identifies letters, words, and other text in an image and extracts the text
    2. Spatial Analysis
        1. Analyzes presence and movement of people in video
            1. What exactly is there in the video
    3. Image Analysis
        1. It contains: Object detection, face detection, categorization, etc.

Using Computer Vision

1. Portal:
    1. Search: Computer Vision
        1. Marketplace: Computer Vision
            1. Creation of Computer Vision Resource
                1. Resource group: cognitive-rg
                2. Region: West Europe
                3. Name: azcourse-vision-mutthoju (unique)
                4. Pricing tier: Free (20 calls per minute, 5000 calls per month)
                    1. Standard (10 calls per second)
                        1. Production will decide which one to select
                5. Check the responsible AI notice acknowledgement
                6. Review + Create
                7. Create
                    1. Deployment is quite short
            2. Go to the resource
                1. Overview
                    1. API type: Computer Vision
                2. Keys and Endpoint
                    1. Two keys
                        1. Copy key 1
                    2. Endpoint
                3. Quick start
                    1. API Console - specialized tool which allows us to try CV API without writing code and any external tool
                        1. Analyze Image (API)
                            1. Region: Select West Europe
                                1. Headers:
                                    1. Ocp-Apim-Subscription-Key: copy and paste the subscription key
                                2. Query parameters
                                    1. visualFeatures: Categories (categories of objects in the image)
                                    2. language: en
                                    3. model-version: latest
                            2. Body:
                                1. Download the file from resources
                                    1. URLs of various images
                                2. Paste the first URL
                                    1. Open it in browser tab
                            3. Send
                                1. Response:
                                    1. Categories: building
                                        1. Probability: 87%
                                    2. Metadata about image:
                                        1. Height
                                        2. Width
                                        3. Format
                            4. Query parameters:
                                1. details: Landmarks (specific landmarks visible)
                            5. Send
                                1. Response:
                                    1. Landmarks: Eifel Tower
                                    2. Probability: 99.8%
                            6. Headers:
                                1. detail: Remove it
                                2. visualFeatures: Description,Tags (no space in between)
                            7. Send
                                1. Response:

1. List of tags: Indicate what we see in the photo:
    1. outdoor
    2. building
    3. sky
    4. tree
    5. monument
    6. ...
2. Description:
    1. Free text: "a tall metal tower with Eifel Tower in the background"
8. We can play with other parameter values
9. Query Parameters:
    1. visualFeatures: Faces
10. Change the URL to a second URL
11. Send
    1. Response:
        1. List of faces
            1. Age
            2. Gender
            3. Coordinates of face rectangle

2. Detect Object API
    1. Select West Europe
        1. Headers:
            1. Ocp-Apim-Subscription-Key: copy paste key
        2. Use the URL of third image (city street)
        3. Send
            1. Response:
                1. A list of objects that CV identified in the photo
                    1. Identified objects & their coordinates

2. We can add AI and ML capabilities to our system without any prior AI or ML knowledge
    1. We just need to know how to use the API

Resources

1. There are lots more APIs
    1. QnA Maker: https://docs.microsoft.com/en-us/azure/cognitive-services/qnamaker/
    2. Anomaly Detector: https://docs.microsoft.com/en-us/azure/cognitive-services/anomaly-detector/
    3. Spatial Analysis: https://docs.microsoft.com/en-us/azure/cognitive-services/computer-vision/intro-to-spatial-analysis-public-preview
2. Cognitive Services can be protected using VNets and Azure AD authentication
    1. Configuring Cognitive Services Virtual Networks
    2. Authenticate requests to Cognitive Services

Section 8: Case Study

Introduction

Introducing our App

Requirements

Component Mapping

Upload Photos Component

Process Photos Component

Registration Component

Retrieve Photos Component

Cost Estimation

Download the Architecture Diagram

Section 9: Conclusion

Conclusion

BONUS: Next Steps